Audio

An audio signal is an analog signal and hence contains infinite signals in a time interval. One, needs to compress this information and form a digital representation. Many format like the `.wav`, `.mp3`, `.flac` exist. All of these do the same thing, but create a digital representation differently.

1. **Sampling Rate:**
   → Sampling is the process of measuring the value of a continuous signal at fixed time steps. The sampled waveform is *discrete*, since it contains a finite number of signal values at uniform intervals. It is also called as the sampling frequency.
   → **The sampling rate is the number of samples taken in 1 second and is measure in Hz. HD Audio has a Sampling rate of 192KHz** .
   → The choice of sampling rate depends upon the highest frequency that can be captured from the signal. This is callde the Nquist limit and is exactly half the sampling rate. The audible frequencies for humans is 8kHz, so 16kHz is more than enough.
   → The sampling rate determines the time interval between successive audio samples, which impacts the temporal resolution of the audio data.

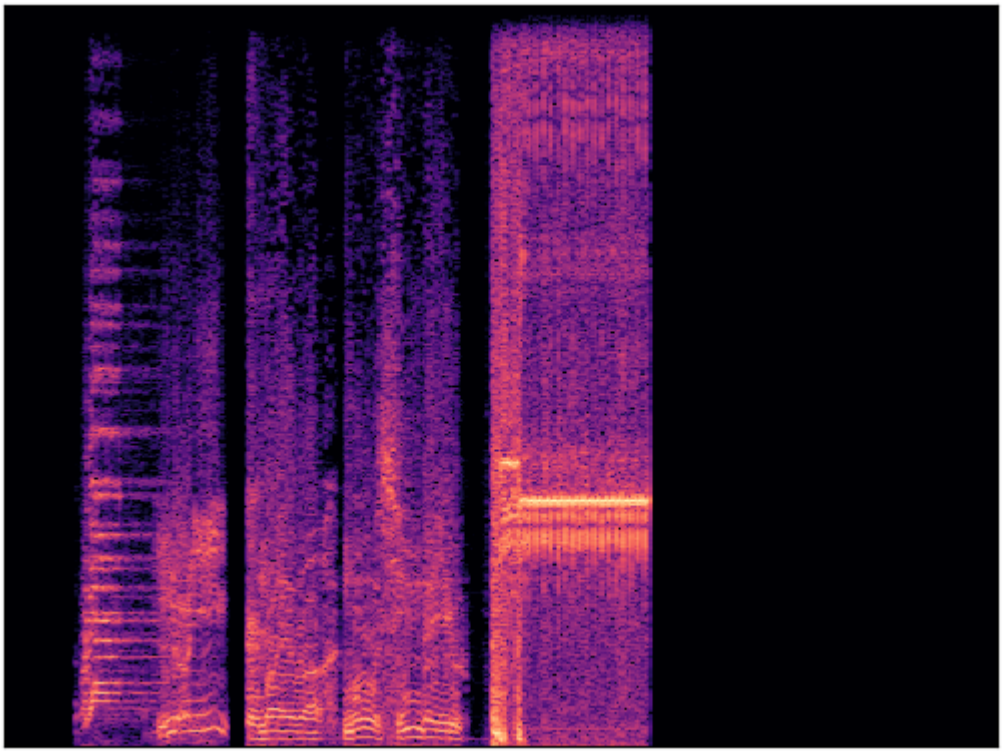
> **NOTE**
>
> It's important to ensure that all audio examples in your dataset have the same sampling rate when working on any audio task. If you plan to use custom audio data to fine-tune a pre-trained model, the sampling rate of your data should match the sampling rate of the data the model was pre-trained on

2. **Amplitude and BIt Depth:**
   → he **amplitude** of a sound describes the sound pressure level at any given instant and is measured in decibels (dB). We perceive the amplitude as loudness.
   → In digital audio, each audio sample records the amplitude of the audio wave at a point in time. The **bit depth** of the sample determines with how much precision this amplitude value can be described. The higher the bit depth, the more faithfully the digital representation approximates the original continuous sound wave.
   → Most common bit depths are 16-bit and 24-bit. Each is a binary term, representing the number of possible steps to which the amplitude value can be quantized when it's converted from continuous to discrete: 65,536 steps for 16-bit audio, a whopping 16,777,216 steps for 24-bit audio
   → The higher the bit depth, the smaller this quantization noise. In practice, the quantization noise of 16-bit audio is already small enough to be inaudible, and using higher bit depths is generally not necessary.
3. **Spectogram:** #todo

4. **Mel-Spectogram:** #todo

Visualizing Audio data

- **Audio as a waveform** or time domain representation
- The **frequency Spectrum** or frequency domain representation. This is computed using fourier transform of the waveform. The higher the bit depth, the smaller this quantization noise. In practice, the quantization noise of 16-bit audio is already small enough to be inaudible, and using higher bit depths is generally not necessary, generally plotted in logarithmic scale.
- In practice, people use the term FFT interchangeably with DFT, as the FFT or Fast Fourier Transform is the only efficient way to calculate the DFT on a computer.
- **Spectogram:** The brighter regions in each bin depicts that the sound data is highly concentrated and the darker regions in each bin depicts the less concentration of sound.



Audio Preprocessing Techniques:

1. **Trimming:** removing the silence from audio using spectogram. This is often done by analyzing short-time energy or spectrogram magnitude and removing frames below a certain threshold.
2. **Remix:** Remixing refers to rearranging segments of audio. In librosa, this often means cutting the signal into intervals (for example, between beats or detected segments) and reordering them. In music tasks, this can be used for creative manipulation or augmentation. In speech tasks, remixing is less common unless you are artificially reordering segments for augmentation or alignment experiments.
3. **Time Strecthing:** Time stretching changes the speed (tempo) of audio without altering its pitch. It makes speech faster or slower while preserving the perceived voice

frequency characteristics. This is widely used for data augmentation in speech recognition.

4. **Normalization:** Normalization adjusts the amplitude of the waveform to a standard level. This can mean scaling the signal so that its peak or RMS energy matches a fixed value. The goal is to reduce variability due to recording volume differences. Without normalization, your model may accidentally associate loudness differences with linguistic patterns, which is undesirable.
5. **Noise Reduction:** Noise reduction attempts to suppress background noise while preserving speech content. Techniques range from simple spectral subtraction to advanced deep learning denoising models. For transcription tasks, especially with real-world or non-native speech, noise reduction can improve clarity, but aggressive denoising may remove subtle phonetic cues. So it must be applied carefully.
6. **Resampling:** Noise reduction attempts to suppress background noise while preserving speech content. Techniques range from simple spectral subtraction to advanced deep learning denoising models. For transcription tasks, especially with real-world or non-native speech, noise reduction can improve clarity, but aggressive denoising may remove subtle phonetic cues. So it must be applied carefully.
7. **Segmentation:** Segmentation involves dividing long audio into smaller chunks. This can be done using silence detection, voice activity detection, or fixed-length windows. Segmentation is crucial for transcription systems because models typically process audio in manageable windows. It also helps align audio with transcripts during training.

Feature Extraction:

It is important to extract features from the audio signal, since it is an unstructured data. The extracted features can be useful for tasks like data analysis, visualization and machine learning model development.

There is a specified module in librosa for feature extraction which comprises of different functionalities for feature extraction which are mentioned below:-

1. :**Chroma features** describe the energy distribution across the twelve pitch classes of Western music (C, C#, D, etc.), independent of octave. Instead of caring about absolute frequency, chroma collapses frequencies into pitch categories. It is very useful for harmonic analysis, chord recognition, key detection, and music similarity tasks. In speech, however, chroma is generally less informative unless you are studying prosody or tonal structure in a musical context.
2. melspectrogram
3. **MFCC (Mel-Frequency Cepstral Coefficients)** are derived from the mel spectrogram through a discrete cosine transform. They compactly represent the spectral envelope, which characterizes the timbre or vocal tract shape. MFCCs are classical features in speech recognition and speaker recognition systems. They reduce dimensionality

while preserving phonetic information, which makes them highly suitable for machine learning.

4. rms #todo
5. spectrum
6. polynomial
7. tonnetz
8. zero crossing rate

One can use Whisper's feature extractor for this process. This can aid to pre-processing as well. Whisper feature extractor pads/truncates a batch of audio examples such that all examples have an input length of 30s. Examples shorter than this are padded to 30s by appending zeros to the end of the sequence (zeros in an audio signal correspond to no signal or silence). Examples longer than 30s are truncated to 30s. Since all elements in the batch are padded/truncated to a maximum length in the input space, there is no need for an attention mask. Whisper is unique in this regard, most other audio models require an attention mask that details where sequences have been padded, and thus where they should be ignored in the self-attention mechanism. Whisper is trained to operate without an attention mask and infer directly from the speech signals where to ignore the inputs.

The second operation that the Whisper feature extractor performs is converting the padded audio arrays to log-mel spectrograms. As you recall, these spectrograms describe how the frequencies of a signal change over time, expressed on the mel scale and measured in decibels (the log part) to make the frequencies and amplitudes more representative of human hearing.

Whisper model has an associated feature extractor and tokenizer, called WhisperFeatureExtractor and WhisperTokenizer respectively. To make our lives simple, these two objects are wrapped under a single class, called the WhisperProcessor. We can call the WhisperProcessor to perform both the audio pre-processing and the text token post-processing.

TLDR: truncation or padding to make all audio lengths same → log-mel spectograms

Models based upon Input and Output:

For audio tasks, the input and/or output sequences may be audio instead of text:

- Automatic speech recognition (ASR): The input is speech, the output is text.
- Speech synthesis (TTS): The input is text, the output is speech.
- Audio classification: The input is audio, the output is a class probability — one for each element in the sequence or a single class probability for the entire sequence.
- Voice conversion or speech enhancement: Both the input and output are audio.

→ **Waveform Input:**

1. Models such as **Wav2Vec2** and **HuBERT** use the audio waveform directly as the input to the model.
2. This raw waveform is first normalized to zero mean and unit variance, which helps to standardize audio samples across different volumes (amplitudes).
3. After normalizing, the sequence of audio samples is turned into an embedding using a small convolutional neural network, known as the feature encoder. Each of the convolutional layers in this network processes the input sequence, subsampling the audio to reduce the sequence length, until the final convolutional layer outputs a 512-dimensional vector with the embedding for each 25 ms of audio.
4. Once the input sequence has been transformed into a sequence of such embeddings, the transformer will process the data as usual.

→ **Spectogram Input:**

1. One downside of using the raw waveform as input is that they tend to have long sequence lengths.
2. Because of this, raw audio waveforms are not usually the most efficient form of representing an audio input. By using a spectrogram, we get the same amount of information but in a more compressed form.
3. Models such as **Whisper** first convert the waveform into a log-mel spectrogram. Whisper always splits the audio into 30-second segments, and the log-mel spectrogram for each segment has shape `(80, 3000)` where 80 is the number of mel bins and 3000 is the sequence length
4. The log-mel spectrogram is then processed by a small CNN into a sequence of embeddings, which goes into the transformer as usual.

→ **Spectogram Output:**

1. For models that generate audio, such as a text-to-speech (TTS) model, we'll have to add layers that can produce an audio sequence. It's very common to generate a spectrogram and then use an additional neural network, known as a vocoder, to turn this spectrogram into a waveform.
2. A so-called post-net, made up of additional linear and convolutional layers, refines the spectrogram by reducing noise. The vocoder then makes the final audio waveform.

ⓘ **NOTE**

If you take an existing waveform and apply the Short-Time Fourier Transform or STFT, it is possible to perform the inverse operation, the ISTFT, to get the original waveform again. This works because the spectrogram created by the STFT contains both amplitude and phase information, and both are needed to reconstruct the waveform. However, audio models that generate their output as a spectrogram typically only predict the amplitude information, not the phase. To turn such a spectrogram into a waveform, we have to somehow estimate the phase information. That's what a vocoder does.

Audio Classification Architecture

#todo

Evaluation metrics

Refer to AI Metrics

- When assessing speech recognition systems, we compare the system's predictions to the target text transcriptions, annotating any errors that are present. We categorise these errors into one of three categories:
  1. Substitutions (S): where we transcribe the **wrong word** in our prediction ("sit" instead of "sat")
  2. Insertions (I): where we add an **extra word** in our prediction
  3. Deletions (D): where we **remove a word** in our prediction
- These error categories are the same for all speech recognition metrics. What differs is the level at which we compute these errors: we can either compute them on the *word level* or on the *character level*.
- Different metrics are:
  1. Character Error Rate
  2. Word Error Rate
  3. Word Accuracy
  4. Inverse Real Time Factor: A model with excellent WER but very low RTFx may not be practical for real-time applications, while a model with slightly higher WER but high RTFx might be more suitable for latency-sensitive use cases."
  5.
- Normalzation #todo

Training and FineTuning:

- **Data collator**:
  1. the data collator takes our pre-processed data and prepares PyTorch tensors ready for the model.
  2. The data collator for a sequence-to-sequence speech model is unique in the sense that it treats the `input_features` and `labels` independently: the `input_features` must be handled by the feature extractor and the `labels` by the tokenizer.
  3. The `input_features` are already padded to 30s and converted to a log-Mel spectrogram of fixed dimension, so all we have to do is convert them to batched PyTorch tensors. We do this using the feature extractor's `.pad` method with `return_tensors=pt`.
  4. Note that no additional padding is applied here since the inputs are of fixed dimension, the `input_features` are simply converted to PyTorch tensors.

5. On the other hand, the `labels` are un-padded. We first pad the sequences to the maximum length in the batch using the tokenizer's `.pad` method.
6. The padding tokens are then replaced by `-100` so that these tokens are **not** taken into account when computing the loss. We then cut the start of transcript token from the beginning of the label sequence as we append it later during training.

- **Load a pre-trained checkpoint:** we need to load a pre-trained checkpoint and configure it correctly for training.
  TLDR: load a dataset → define data colator → evaluation metric → training arguments

Transcription Models:

- Whisper is a strong pre-trained model for speech recognition and translation. Compared to Wav2Vec2, it has higher transcription accuracy, with outputs that contain punctuation and casing
- One can use chunking for longer audio in whisper.
- There is also varying performance across different accents and dialects of certain languages, including lower accuracy for speakers of different genders, races, ages or other demographic criteria.

Whisper:

- trained on extremely noisy imperfect data

Reference:

1. https://huggingface.co/learn/audio-course/en/chapter0/introduction
2. https://www.kaggle.com/code/vuppalaadithyasairam/audio-pre-processing-tutorial-in-python